When, Not Whether: What a Procedural Graph Actually Contributes to Online Mistake Detection in Egocentric Video

[[AUTHOR LIST]]
[[AFFILIATIONS]]
[[CONTACT EMAIL]]

Omit for ARR anonymous submission; restore at camera-ready.

Abstract

Detecting that someone has just made a mistake while following a procedure requires more than spotting an unusual moment: the same action can be correct at one point in a recipe and an error at another. We ask whether an explicit, machine-readable representation of the correct procedure, supplied to a vision–language model as part of its prompt, is enough to turn a general-purpose VLM into an online procedural mistake detector. Our method, GRAPHWALK, walks a recipe's task graph as a state machine driven by the model's own window-by-window judgements: at each causal 4-second window the model receives the frames together with the step it should be seeing, the admissible next steps, and a summary of its own recent decisions, and returns a structured judgement: deviation or not, and of what type. On the complete CaptainCook4D test split (109 recordings, 23,696 windows, no subsetting), this reaches 0.482 binary F1 against 0.150 for the identical model prompted zero-shot, and matches a reproduced recent online detector re-evaluated on the identical windows (+0.005 F1, CI [-0.060, +0.059]).

We then ask what the graph is responsible for, and the answer is not what we expected. Removing it at inference, and removing it from training as well, leaves binary F1, AUC and macro-F1 statistically unchanged (0.460 vs 0.482, paired CI [-0.024, +0.064]): a model that never sees a procedural graph detects mistakes just as well. What the graph does change, in both controls and with intervals excluding zero, is *earliness* — early detection 0.806 against 0.697 and 0.673, at the same event coverage. We further show why the detection effect is absent: our step pointer advances once per window and therefore saturates, leaving 66% of all windows conditioned on the final step. The contribution of procedural conditioning here is to tell an online detector *when* to look, not *whether* something is wrong — and that conclusion is reached against a pointer we show to be broken, which bounds it further.

1 Introduction

A person following a recipe washes a cucumber, then slices it. Reverse those two actions and nothing anomalous has happened in any visual sense — both actions are ordinary, both are in the recipe — yet the second ordering is a mistake. This is the defining difficulty of *procedural* mistake detection, and it is what separates the problem from video anomaly detection. Correctness is not a property of a moment; it is a property of a moment *relative to a plan*.

Systems that can make this judgement online, as the action unfolds rather than in hindsight, would underwrite assistive AR guidance, industrial training, and surgical or laboratory checklists. The requirement to be causal is not a convenience: a detector that needs the rest of the video cannot warn anyone.

Recent work has attacked the problem from two directions. One line builds procedural structure explicitly, learning or supplying task graphs and flagging deviations from the expected ordering (Flaborea et al., 2024; Seminara et al., 2024; Lee et al., 2024); these methods reason over the plan, but the perceptual front end is usually an action recogniser, so the plan can only be consulted through a fixed action vocabulary, and the output is typically a binary flag. A second line applies vision–language models directly to the video (Storks et al., 2025; Spurio et al., 2026), gaining open-vocabulary perception and human-readable rationales, but conditions the model on generic instructions or free text rather than on the procedure's structure, and so has no representation of *where in the plan the person should currently be*.

We ask whether the two can simply be joined: give a VLM the procedural graph, in the prompt, at every window. The appeal is that the plan and the percept meet in the same medium — language — so no fixed action vocabulary is needed, and the class set lives in the prompt rather than in a classifier head, which means the label space can change without retraining the architecture.

Two design choices shape how we do this, and we describe them as choices rather than as demonstrated mechanisms. First, the graph is *walked* rather than merely attached: instead of placing the whole step list in the context, we advance an expected-step pointer whenever the model reports no deviation, so the graph behaves as a state machine whose transitions are the model's own judgements. We did not run a static-dump arm, so we make no claim about how much this specific choice contributes. Second, the model's self-reported confidence is unusable. Zero-shot it takes two distinct values; after fine-tuning it collapses to a single constant, because the supervision targets contain a constant. Reading confidence from the generation logits instead — the probability mass on the boolean token

emitted for the deviation field — is what makes any ranking or thresholding possible: with a constant field there is nothing to rank, and the resulting score reaches AUC 0.636.

We evaluate on CaptainCook4D (Peddi et al., 2024), which supplies gold task graphs and temporally bounded error annotations, on the complete test split with no subsampling. Our contributions:

- **A negative result on the load-bearing claim, obtained against a pre-registered reading.** Graph conditioning does not improve detection. Across two controls — graph removed at inference, and removed from training as well — binary F1, AUC and macro-F1 are statistically unchanged. The 0.332 gain over zero-shot belongs to fine-tuning. We state the interpretation of all five possible outcomes in Section 6.2 *before* reporting any of them.
- **A positive result that replicates: the graph buys earliness.** Early detection drops from 0.806 to 0.697 and 0.673 in the two controls, both intervals excluding zero, at unchanged event coverage. The plan tells the detector when a deviation is due, not whether a window looks wrong.
- **A diagnosis of why the detection effect is absent.** The step pointer advances once per window while a recipe step spans many, so it reaches the final step at a median 16% of the way through a recording: 66% of all windows carry a constant graph context. The ablation therefore measures a saturating pointer, which we report rather than present the null as a property of procedural conditioning.
- **A detector and protocol reported at full scale.** 0.482 binary F1 on all 109 test recordings, parity with a reproduced ICCV'25 detector (Patsch et al., 2025) under a matched protocol and a non-oracle threshold, every number with clip-level bootstrap intervals and a decision layer selected on validation and applied to test once.
- **An evaluation protocol reported at full scale with uncertainty**: every number is computed on all 109 test recordings and reported with clip-level bootstrap intervals, with the decision layer selected exclusively on validation and applied to test exactly once.

2 Related Work

Online procedural mistake detection. The dominant formulation routes video through an action recogniser and reasons about mistakes over the resulting symbol stream. PREGO (Flaborea et al., 2024) pairs an online action-recognition branch with a symbolic reasoner that predicts the expected next action, flagging a mistake when recognised and expected disagree; TI-PREGO (Plini et al., 2024) strengthens the reasoner with chain-of-thought and in-context learning. EgoPER (Lee et al., 2024) learns step prototypes contrastively from error-free video and flags departures from them. ESTANet (Lee et al., 2026) drops plan reasoning altogether, running an ensemble of differently-configured action detectors and flagging frames on which they disagree. MistSense (Patsch et al., 2025), our baseline, detects procedural and execution mistakes online from RGB and hand-pose features and queries an LLM for an explanation after a mistake fires.

Two properties are shared across this line and motivate our formulation. First, the plan is consulted only through a fixed action vocabulary, so what the system can notice is bounded by what its recogniser was trained to name. Second, the output is a binary flag: none of these methods predicts a mistake *type*. Our formulation asks for the type and the onset as well, which is why we report per-class F1 with support rather than detection alone.

Task graphs as procedural representations. Task graphs encode a partial order over key-steps and have become the standard human-readable representation of a procedure. Seminara et al. (2024) make graph construction differentiable, learning edge weights by direct maximum likelihood from observed action sequences, and show the learned graphs improve downstream online mistake detection.

Their relationship to this work needs stating precisely, because they also use CaptainCook4D and a reader will reasonably expect a head-to-head number. They use CaptainCook4D for *task-graph generation* only; their mistake-detection experiments are on Assembly101-O and EPIC-Tent-O, and operate on action sequences — ground-truth or predicted — rather than on video. A direct comparison on our benchmark therefore does not exist to be made: the two systems consume different inputs and are evaluated on different datasets for this task. The efforts are complementary in a concrete sense. They study how to *build* a graph and assume a symbolic action stream at detection time; we take the graph as given and study how to *consume* one inside a VLM's context directly from pixels. Pairing learned graphs with graph-conditioned visual reasoning is the natural next step and one we do not take here.

Vision–language models for procedural video. A second line applies VLMs to the video directly. Storks et al. (2025) require the model not merely to detect a procedural mistake but to justify it, and contribute NLI-based metrics for the coherence of the generated rationale. Spurio et al. (2026), contemporaneous with this work, post-train a video-language model for mistake detection with a task-specific reward and report strong generalisation to unseen procedures. Both establish VLMs as a viable front end and both obtain open-vocabulary perception and human-readable explanation, which the recogniser-based line cannot. Neither conditions the model on an explicit representation of the procedure's structure or on where in that structure the person currently is. That gap is what this paper occupies: we supply the graph in the prompt and advance a step pointer with the model's own judgements, so the plan and the percept meet in the same medium. We should be equally clear about the direction we do *not* extend: Storks et al. (2025) supervise and evaluate the generated justification, whereas our targets are built from labels alone and our fine-tuned model's free-text output is degenerate (Section 4). On explanation their setting is the stronger one, and combining their justification objective with our graph conditioning is the obvious thing neither

paper has done.

Distinction from video anomaly detection. Online video anomaly detection asks whether a moment is unusual, and recent work such as MoniTor (Yang et al., 2025) brings LLM instruction and streaming temporal state to that problem. The tasks should not be conflated. Anomaly detection scores departures from a distribution of normal appearance; procedural mistake detection scores departures from a *plan*. Slicing a cucumber is never anomalous in a cooking video, and is a mistake precisely when the recipe required washing it first. The distinguishing evidence is not in the frames but in their relation to the procedure, which is why an explicit plan representation is the operative ingredient here and not merely a source of extra context.

Benchmarks. CaptainCook4D (Peddi et al., 2024) provides 384 egocentric recordings of 24 recipes with fine-grained, temporally bounded error annotations and gold task graphs; the combination of typed errors, temporal bounds, and shipped graphs is what makes it the appropriate benchmark for this study. EgoOops (Haneji et al., 2025) annotates mistakes against procedural texts but is roughly an order of magnitude smaller. Assembly101 (Sener et al., 2022) and EPIC-Tent (Jang et al., 2019) supply the Assembly101-O and EPIC-Tent-O online mistake-detection splits used by much of the prior work above; HoloAssist (Wang et al., 2023) annotates mistakes within instructor–performer dialogue; and Ego4D (Grauman et al., 2022) is the large-scale egocentric corpus underpinning much of this area. None of these ships gold task graphs, which is the binding constraint on evaluating graph-conditioned methods across datasets and the reason our study is single-benchmark (Section 7).

3 Problem Formulation

A recording V of a known task T is tiled into contiguous windows $w_1, \ldots, w_N$ of fixed length, processed in temporal order. The task's correct procedure is given as a graph $G_T = (S, E)$ whose nodes S are steps carrying a natural-language description and whose directed edges E encode precedence.

At window w_i the detector may condition only on $w_{\leq i}$ and on G_T — it never sees the future. It emits a binary deviation decision $\hat{y}_i$, a mistake type $\hat{c}_i$ from a fixed vocabulary $\mathcal{C}$, and a score $s_i \in [0, 1]$. From the resulting stream we additionally derive an onset estimate: the timestamp at which the system first asserts an ongoing deviation, compared against the start of the earliest gold error event on the recording.

This formulation is deliberately stricter than the binary online setting used by most prior work: the system must name the error, not only notice it, and it is scored on *when* it noticed.

4 Method

4.1 Graph representation and graph state

Each recipe's gold DAG is converted into a typed graph of step nodes with precedence edges; the dataset's START and END sentinels are retained as non-executable nodes so every edge endpoint remains valid, and dangling endpoints are rejected at load time. The 24 recipe graphs range from 9 to 27 steps (mean 16.6).

The state carried across windows is a single pointer σ_i into S. It begins at the first step and advances by one whenever the model reports *no* deviation at w_i, clamped at the last step. A persistent deviation therefore holds the pointer in place — the system waits for the person to come back to the plan rather than marching on without them. This is the only piece of inter-window state the method carries besides a three-window judgement history.

4.2 Graph-conditioned prompting

At each window the model receives the sampled frames together with a fixed instruction template filled with: the task name; the current expected step σ_i and its description; the set of valid next steps read off the outgoing precedence edges; the objects, states, and permitted variations associated with the step; and a three-window summary of its own recent judgements rendered as (observed action, deviation flag, predicted class). The template states all class definitions explicitly, including one-line glosses for the distinctions most easily confused, and instructs the model to answer with JSON only.

What the conditioning actually contains. We state plainly what a reader would otherwise have to discover: although our representation supports objects, states, and permitted variations, the gold graphs shipped with the benchmark populate *only* steps and precedence edges. For all 24 recipes the object, state, and variation fields are empty. Graph conditioning as evaluated in this paper therefore means *step description, valid next steps, and step position* — and nothing more. That the effect reported in Section 6 is obtained from this much alone is, if anything, the more interesting reading; but it bounds the claim, and it is why we treat richer graph content as future work rather than as an established ingredient.

4.3 Structured output and defensive parsing

The requested answer has nine fields: observed action, observed objects, matched step, deviation flag, mistake class, confidence, an uncertainty reason drawn from a closed vocabulary, a one-to-two sentence rationale, and a brief expected-versus-observed delta. The JSON object is extracted from the generated text; the mistake class is validated against $\mathcal{C}$ and the uncertainty reason against its vocabulary, with out-of-vocabulary values coerced to the safe default and confidence clamped to $[0, 1]$. A total parse failure falls back to a no-deviation prediction, and the raw text is retained. No parse failures occurred in any reported run.

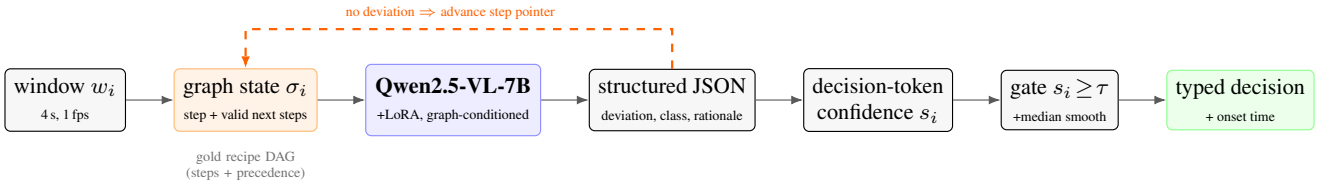


Figure 1: GRAPHWALK. Each causal window is judged against the recipe's task graph at the position the system currently expects. The dashed edge is the mechanism the method turns on: the graph is *walked* as a state machine whose transitions are the model's own no-deviation judgements, so a persistent deviation holds position rather than marching the plan forward without the person. The confidence that gates the decision is read from the model's generation logits, not from the confidence field it writes (Section 4.4).

Only two fields carry information after fine-tuning. The schema is richer than what the fine-tuned model actually uses, and we state this plainly because the output format invites the opposite assumption. Our supervision targets are constructed from the window labels alone: the free-text fields are filled with fixed placeholder strings and the confidence with a constant. The model reproduces those constants exactly. On a probe of 76 consecutive test windows the rationale, observed action, observed objects, expected-versus-observed delta and uncertainty reason each took a *single* value, and the verbalised confidence was constant at 0.9, while the decision-token probability (Section 4.4) ranged 0.020–0.945 over the same windows. **The fine-tuned system is therefore a typed detector, not an explainer**; its informative outputs are the deviation flag, the mistake class, and the decision-token score. Supervising the free-text fields — the benchmark ships free-text error descriptions that could serve as targets — is left to future work, and we make no claim about explanation quality anywhere in this paper.

4.4 Decision-token confidence

The model's verbalised confidence field is not a usable signal. Prompted zero-shot it takes only two distinct values and is statistically indistinguishable between true and false positives (0.859 vs. 0.863). After fine-tuning it is strictly worse: our supervision targets carry a constant confidence, and the model reproduces that constant on every window. A constant score makes any threshold rule inert by construction.

We therefore read the score from the generation logits. Generation is run with per-step scores retained; the emitted token sequence is walked to the `deviation` field and the first subsequent boolean token located. Its softmax probability is taken as s_i — used directly when the model emitted *true* and complemented when it emitted *false*. If the boolean cannot be located the fallback is the geometric mean of the emitted tokens' probabilities, which still varies per window.

This is a per-window signal with the correct polarity, and Section 6 shows it is an informative *ranker* (AUC 0.636). We are careful not to claim more: it is not calibrated, and we report calibration error alongside every result so that this is visible rather than buried.

4.5 Completion-only LoRA fine-tuning

We adapt Qwen2.5-VL-7B-Instruct (Bai et al., 2025) with LoRA (Hu et al., 2022) of rank 32, scaling 64, dropout 0.05, applied to both attention (query, key, value, output) and MLP (gate, up, down) projections — 95M trainable parameters, 1.13% of the model. The objective is ordinary causal language-modelling cross-entropy computed *only over the answer tokens*, with prompt and image tokens masked out.

The masking is not a detail. Computing the loss over the full input — roughly 1850 prompt and image tokens against about 71 answer tokens — drowns the training signal, and in our runs left the loss plateaued near 6.1 without learning. With completion-only masking the loss falls immediately and converges to ≈ 0.017.

One consequence of this design is worth stating for a likely reviewer question: the mistake taxonomy is never compiled into a classifier head. It lives in the prompt and in the targets, so the class set can be changed without touching the architecture.

4.6 Decision layer

Per-window scores become decisions through a small, fully specified layer: a window fires when $s_i \geq \tau$; a fired window keeps its score and a rejected one is zeroed; the resulting stream is median-smoothed with width k; and the binary decision is that the smoothed score is positive. The typed stream is gated in parallel — a rejected window is relabelled `no_mistake` regardless of what the model called it — so the typed comparison across configurations is genuine rather than an artefact of shared intermediates.

Onset localisation deliberately uses a stricter rule than the binary metric: the predicted onset is the timestamp of the first run of at least m consecutive fired windows. A recording whose event never fires scores zero earliness and is excluded from the onset-error mean, with the detected count reported so that the denominator is auditable.

$(\tau, k, m, \text{gate})$ are selected on the validation split alone — τ, k and the gate mode by validation binary F1, m by validation early-detection score at the chosen operating point — then frozen and applied to test exactly once. The selected configuration is $\tau = 0.45$, $k = 5$, $m = 1$. Each arm receives its own validation-tuned layer, so no arm is advantaged by threshold placement.

5 Experimental Setup

5.1 Benchmark

CaptainCook4D (Peddi et al., 2024) contains 384 egocentric recordings of 24 cooking recipes, about 94.5 hours, with 5700 step segments of which 1964 carry at least one error (2574 error instances), and gold task graphs per recipe. We use the official split of 213 train / 62 validation / 109 test recordings. Evaluation covers 14,129 validation and 23,696 test windows, with zero recordings dropped and zero graphs missing.

5.2 Taxonomy and its structural limit

We adopt the benchmark's native error taxonomy minus the `Other` catch-all (8 instances), giving eight classes: `no_mistake`, `order_error`, `technique_error`, `preparation_error`, `measurement_error`, `missing_step`, `timing_error`, `temperature_error`. Steps carrying several error tags are reduced by a documented priority in which structural errors outrank execution errors.

One limitation is structural rather than empirical and must be read alongside every typed result: `missing_step` errors are exactly the non-localisable annotations. A step that never happened produces no video window, so a windowed detector cannot emit the class — its test support is 2 windows and its F1 is 0.000 in every row. In practice the method covers 7 of 8 classes, and we report macro-F1 both with and without it. Sixty-nine non-localisable segments in the test split are unrecallable by any windowed metric for the same reason. One further exclusion is small but worth stating: a single step carrying only the `Other` tag is dropped from the typed metrics, since that catch-all is outside our label set. No recording is excluded, and the binary metrics use every window.

5.3 Windowing and protocol

Windows are 4 seconds long, sampled at 1 frame per second, with a 4-second stride — contiguous and non-overlapping. A window's ground truth is the annotation whose span contains the window's midpoint. Windows are emitted while a full window fits inside the recording, so a trailing remainder shorter than one stride is left unsampled.

Every result in this paper uses the **online causal** protocol. We do not report an offline (full-video) variant. Earlier versions of this work reported one, and we drop it deliberately rather than silently: in that implementation the expected step index was tied to the window index, so it saturated at the final step for any recording longer than its graph — which is most of them. The offline condition therefore measured weaker graph conditioning rather than the benefit of non-causal access, and comparing it against the online rows would have been misleading. Online is also the setting the task is motivated by: a detector that needs the rest of the video cannot warn anyone.

5.4 Metrics

Binary F1 on the mistake-vs-not decision; macro-F1 and per-class F1 reported with support, both including and excluding `missing_step`; AUC-ROC over the raw ungated scores, which is threshold-free and therefore comparable across rows; precision and recall; expected calibration error and Brier score over the raw per-window scores; and an early-detection score (EDS) with the mean absolute onset error in seconds. Per-class F1 scores a never-predicted class as 0.0 rather than dropping it from the average.

Uncertainty. Every headline metric carries a 95% percentile bootstrap interval over 1000 resamples with recordings as the resampling unit (seed 0), since windows within a recording are strongly correlated. We report a paired bootstrap over the same resamples for the difference between arms, and a McNemar test over window decisions as a secondary check.

5.5 Baseline

We reproduce MistSense (Patsch et al., 2025) end to end and retrain its Stage-1 detector on CaptainCook4D so that a comparison exists on the same data. Two deviations are stated for the record: we substitute an ungated LLM (OPT-2.7b) and CLIP-ViT-L/14 for the paper's gated components; both affect only the captioning stages, not the detector we compare against. The reproduced detector scores 0.495 binary F1 on the benchmark under its own native per-frame protocol with a threshold swept on the test set. That threshold selection is an oracle and the protocol is not ours, so we do not treat that number as a comparison; We therefore report a protocol-matched row instead: the baseline's per-frame probabilities are max-pooled onto our identical 23,696 windows — a window counts as a mistake if any moment inside it does — and its decision layer is tuned on validation and frozen, exactly as ours is (selected $\tau = 0.7$). Coverage is complete: all 109 test recordings, no missing probability dumps. This is the only baseline row we treat as comparable.

For completeness we also report the baseline under its own native per-frame protocol with the threshold selected on validation rather than on test: it reaches 0.452 F1 at $\tau = 0.7$, against 0.456 for the best threshold chosen on test. On that protocol, then, honest threshold selection costs only 0.005 — the larger oracle effect noted above belongs to the coarse 872-window protocol under which the 0.495 figure was obtained, not to threshold selection in general.

5.6 Implementation

Qwen2.5-VL-7B-Instruct in bfloat16 on a single RTX PRO 6000 (96 GB), greedy decoding (temperature 0) with a 512-token generation cap, global seed 42. For fine-tuned evaluation the LoRA adapter is merged into the base weights, so the fine-tuned and zero-shot runs differ only in weights and are otherwise the same execution path. Training uses a class-balanced 10,000-window subset of the training recordings (4951 mistake

/ 5049 normal) drawn at the same window length, stride and frame rate as evaluation, learning rate 2×10^{-4} with cosine schedule and warmup, gradient accumulation 8, two epochs, about three hours. The window index is built without decoding pixels and frames are decoded lazily, which is what makes the dataset's $\sim$90k candidate windows tractable.

6 Results

6.1 Graph-conditioned fine-tuning

Table 1 gives the headline comparison. GRAPHWALK reaches 0.482 binary F1 against 0.150 for the identical model, identical prompt, identical recordings and windows, prompted zero-shot. The paired clip-level bootstrap puts the difference at 0.332 with a 95% interval of [0.246, 0.409], and a McNemar test over the 23,696 window decisions gives $p = 2.8 \times 10^{-8}$. Because both rows come from the same recordings, windows, and protocol, and each received its own validation-tuned decision layer, this is a like-for-like comparison.

The ranking quality moves further than the F1: AUC over raw scores goes from 0.480 — indistinguishable from chance — to 0.636. This matters for reading the zero-shot row correctly. Its 0.150 F1 is not evidence that the zero-shot model detects anything; it is what a validation-tuned threshold can extract from a near-chance ranker, and its very high EDS reflects indiscriminate early firing rather than accurate onset detection. Zero-shot Qwen2.5-VL is, on this task, effectively a non-detector.

6.2 Isolating the graph

The comparison in Section 6 establishes that graph-conditioned fine-tuning works. It does not, on its own, establish that the *graph* is what works, because fine-tuning and graph conditioning move together between the two rows. A model that improved purely by adapting to the benchmark's visual domain would produce the same table.

We therefore run four controls. All four hold the decision layer of the main row fixed — the same τ, smoothing width, persistence and gate mode, frozen on validation before any of these runs existed — so the only variable is what the model is told about the procedure. Three are inference-only and reuse the fine-tuned adapter; one retrains.

R5 — graph stripped at inference. The fine-tuned model is evaluated with the graph block removed from the prompt. This measures how much of the model's behaviour depends on the graph being present *at the moment of judgement*, as opposed to having been present during training. It is the cheapest control and the one that most directly addresses the attribution question.

R6 — graph-free training and evaluation. A second adapter is trained from the base model under identical hyperparameters, seed, and data, but with the graph-free prompt throughout, then evaluated graph-free. R5 and R6 together separate two effects that a single graph-off row would conflate: a drop in R5 alone could mean the model genuinely uses the graph, or merely that it was trained on a prompt format it is no longer being given. If R6 recovers what R5 loses, the effect was format mismatch; if R6 does not recover it, the graph carries information the model cannot obtain otherwise.

R7 — deranged graph. Every recording is conditioned on a *different* recipe's graph, under a fixpoint-free permutation with a fixed seed, so the prompt has exactly the same structure, length and register as the main arm while its content is wrong. This separates the value of the plan's content from the value of having any structured scaffold in the context. A system that performs as well on wrong graphs as on right ones is not using the plan; it is using the presence of a plan-shaped block.

R10 — static dump. The full step list for the correct recipe is placed in the prompt, but the model is not told where in that list the person should currently be. This isolates the walked step pointer (Section 4) from the plan itself, and is the control that decides whether *walking* the graph is a mechanism or merely an implementation choice.

How these rows should be read. We state the interpretation before reporting the numbers, so that the reading is not selected after seeing them. Writing $F(\cdot)$ for binary F1 on the frozen decision layer:

- If $F(\text{R4}) \approx F(\text{R5}) \approx F(\text{R6})$, graph conditioning contributes nothing measurable and the honest headline is that fine-tuning, not the graph, produces the result. The paper's title claim would not survive, and we would report it as such.
- If $F(\text{R4}) > F(\text{R5})$ but $F(\text{R6}) \approx F(\text{R4})$, the drop is prompt-format mismatch rather than lost information: the model can do as well without the graph provided it was trained that way.
- If $F(\text{R4}) > F(\text{R6})$ with a clip-level interval excluding zero, graph conditioning carries information that graph-free training cannot recover, and the central claim holds.
- If additionally $F(\text{R4}) > F(\text{R7})$, the plan's *content* matters and not merely the presence of structure. If $F(\text{R7}) \approx F(\text{R4})$, the effect is a prompt-shape artefact and should be described that way however favourable R5 and R6 look.
- If $F(\text{R4}) > F(\text{R10})$, the walked pointer is doing work beyond supplying the plan, and the mechanism described in Section 4 is supported. If not, the walk is an implementation detail and we say so.

Every comparison uses the paired clip-level bootstrap of Section 5, with recordings as the resampling unit; we treat a difference whose 95% interval contains zero as unresolved rather than as evidence of equivalence, given the interval widths involved.

System	binary F1	P	R	macro F1	macro F1$^-$	AUC	ECE	EDS	onset MAE (s)
Zero-shot Qwen2.5-VL, graph prompt	0.150 [0.127, 0.172]	0.264	0.105	0.108 [0.100, 0.115]	0.123	0.480 [0.447, 0.512]	0.527	0.985 [0.971, 0.996]	64.4
GRAPHWALK (graph prompt + LoRA)	0.482 [0.392, 0.558]	0.407	0.589	0.185 [0.161, 0.208]	0.212	0.636 [0.563, 0.698]	0.490	0.806 [0.705, 0.892]	60.4
GRAPHWALK w/o graph at inference (R5)	0.467 [0.383, 0.536]	0.419	0.527	0.191 [0.166, 0.211]	0.218	0.631 [0.564, 0.688]	0.473	0.697 [0.591, 0.801]	81.5
GRAPHWALK trained *and* tested graph-free (R6)	0.460 [0.381, 0.534]	0.453	0.467	0.186 [0.165, 0.206]	0.213	0.629 [0.560, 0.692]	0.451	0.673 [0.565, 0.778]	79.6
MistSense (Patsch et al., 2025), protocol-matched	0.477 [0.396, 0.557]	0.325	0.893	0.103 [0.096, 0.108]	0.118	0.585 [0.539, 0.630]	0.454	0.991 [0.985, 0.997]	60.9
GRAPHWALK w/ deranged graph (R7)	**[[WRONG-GRAPH-CONTROL]]**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**
GRAPHWALK w/ static step dump, no position (R10)	**[[STATIC-DUMP-CONTROL]]**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**	**TBA**

Table 1: Online causal detection on the **complete** CaptainCook4D test split: all 109 recordings, 23,696 windows, no subsetting. Each arm uses its own decision layer selected on validation and applied to test once. Brackets are 95% clip-level bootstrap intervals (1000 resamples, seed 0). macro F1$^-$ excludes the structurally unreachable `missing_step` class. AUC is computed on raw ungated scores and is therefore threshold-free. ECE is reported for every row because the decision-token score is informative but uncalibrated (Section 4.4); lower is better, and the baseline is the best-calibrated row. Binary F1 and the localisation columns use different firing rules by design (Section 4.6).

Row	Condition	Plan content	Position
R4	Graph-conditioned (main)	yes	yes
R10	Static dump of all steps	yes	no
R7	Deranged (wrong recipe) graph	wrong	yes
R5	Graph stripped at inference	no	no
R6	Graph-free training and test	no	no

Table 2: What each control varies. *Plan content* is whether the prompt carries this recipe's steps; *position* is whether it also says where in those steps the person is expected to be. R5 and R6 differ not in conditioning but in whether the model was ever *trained* with a graph.

Results: R5, graph stripped at inference. Removing the graph from the prompt costs +0.015 binary F1 (0.482 → 0.467), with a paired clip-level interval of [-0.003, +0.035] that *contains zero*. The threshold-free ranking is likewise unchanged (AUC 0.636 → 0.631, interval containing zero), and macro-F1 does not drop at all (0.185 → 0.191). By the criteria set out above, we cannot claim a detection-quality effect from graph conditioning at inference: on everything the detection literature reports, the graph-conditioned and graph-stripped models are indistinguishable on this benchmark.

One quantity does move, and it moves decisively. Early detection falls from 0.806 to 0.697 — a paired difference of +0.109 with interval [+0.022, +0.194], excluding zero — and mean onset error rises from 60.4 s to 81.5 s. The two systems detect the *same* number of events (57 of 61 in both cases); the graph-conditioned model detects them *earlier*. This is a coherent reading of the mechanism: the expected-step pointer tells the model where in the procedure a deviation should be looked for, which is information about *when* to fire rather than about *whether* a window is anomalous.

We flag plainly that this is not the result the framing of this paper anticipated, and we have not rewritten the framing to fit it. The pre-registered reading applies: on detection quality, the honest statement is that graph conditioning at inference contributes nothing measurable, and the zero-shot-to-fine-tuned gap of 0.332 is attributable to fine-tuning. What the graph demonstrably buys is earliness, which for an online detector is not a secondary property — but it is a different claim from the one the title makes.

Why the graph does so little: the pointer saturates. The null result on detection has a concrete mechanical cause, which we found by instrumenting the step pointer rather than by speculating. The pointer advances by one step for every window the model judges non-deviant — but a recipe step spans many 4-second windows, so the pointer moves through the graph roughly an order of magnitude faster than the person does. It therefore reaches the final step early and clamps there. Across the test split it first reaches the last step at a median of 16% of the way through a recording, and 66% of *all* windows are conditioned on the final step (per-clip median 78%; 77 of 109 recordings spend the majority of their windows saturated).

For two-thirds of the windows, then, the "graph context" is a constant string naming the last step of the recipe, carrying no information about where the person actually is. That a graph-conditioned and a graph-stripped model score identically is exactly what one should expect under that condition. R5 does not show that procedural structure is useless for this task; it shows that *this implementation of the pointer stops delivering structure after the first sixth of a recording*. Notably, this is the same saturation failure we identified in the offline protocol and rejected the protocol for (Section 5) — it is present in the online protocol too, for a different reason, and we did not notice it until the ablation forced the question.

And the training signal was worse. Instrumenting the pointer led us to check the corresponding assignment during fine-tuning, where we found a second and larger problem. Training does not use the advancing pointer at all: it assigns each window the step at its own index, clamped to the last step. A recipe has 9–27 steps and a recording runs for minutes, so positional variation exists only in the first ∼17 windows — about a minute — of each recording. Across the training pool, 93% of windows carry the final step as their graph context (per-recording median 92%).

Two consequences follow, and both bear on how the ablation should be read. The model was fine-tuned on a conditioning signal that was constant for 93% of its examples, so it had almost no opportunity to learn to use step position even in principle. And the rule differs between training and evaluation, so what positional signal did exist at test time was not the signal the model was trained on. The ablation result is therefore not evidence that a VLM cannot use procedural structure; it is

evidence that this system never delivered procedural structure to be used.

We report this because it is the most likely reason for the null and because it was our own defect, found only when the ablation forced us to look. The correction is not a tuning change: the step assignment must track a person's progress through the procedure — advancing on evidence that a step has completed — and it must be the same rule in training and evaluation. That requires re-training, so it is future work rather than a revision of these results. Until it is done, this paper's evidence about graph conditioning speaks to a degenerate delivery of the graph rather than to procedural conditioning in general, and we would not want the negative result read more broadly than that.

Results: R6, trained and tested without the graph. R6 removes the remaining explanation. A second adapter, trained from the base model on graph-free prompts and evaluated graph-free, reaches 0.460 binary F1 against 0.482 for the graph-conditioned model. The paired difference is +0.022 with a 95% interval of [-0.024, +0.064], containing zero; AUC and macro-F1 likewise show no effect. A model that never saw a procedural graph, at any point in training or inference, is indistinguishable from GRAPHWALK on every detection metric we report.

This is the first of the five patterns registered above, and we report it as such: **on detection quality, graph conditioning contributes nothing measurable, and the large gain over zero-shot — 0.332 F1 — is attributable to fine-tuning**. The title this paper began with claimed the graph; that claim does not survive its own ablation, and we have changed the title rather than qualify it in a footnote.

What does survive: the graph changes *when*, not *whether*. One effect is present in both controls and in the same direction. Early detection falls from 0.806 to 0.697 without the graph at inference (paired +0.109, CI [+0.022, +0.194]) and to 0.673 without it in training as well (paired +0.133, CI [+0.054, +0.224]). Both intervals exclude zero. Mean onset error moves correspondingly, 60.4 s against 81.5 s and 79.6 s. The three systems detect essentially the same events — 57, 57 and 56 of 61 — so this is not a difference in what is found but in how soon.

We take the replication across two independent controls seriously: the effect survives removing the graph at inference and removing it from training altogether, with non-overlapping mechanisms of failure. It is also the effect the method's design predicts, since an expected-step pointer supplies information about where in a procedure a deviation is due rather than about whether a window looks wrong. For an online detector, earliness is not a minor axis — a warning that arrives twenty seconds sooner is the difference between interrupting a mistake and describing one. But it is a narrower claim than the one this work set out to make, and the margin is measured against a pointer that we have shown saturates for two-thirds of all windows.

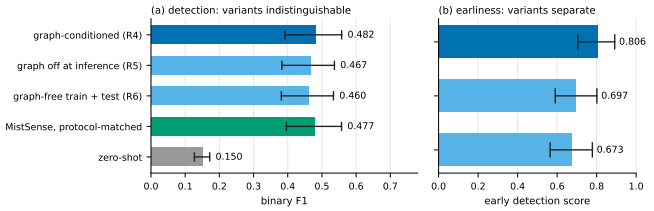


Figure 2: Where the effect comes from. **(a)** On detection, every conditioning variant lies inside every other's interval, and the whole cluster sits far above zero-shot: the distance is fine-tuning, not the graph. **(b)** On earliness the variants separate, and the paired differences against R4 exclude zero. Only the three conditioning variants appear in (b): MistSense (0.991) and zero-shot (0.985) score highest on this metric because they fire on almost every window, so plotting them here would invite the misreading the text warns against — early detection is interpretable only alongside precision. Bars are clip-level bootstrap intervals (1000 resamples, seed 0).

Results: R7 and R10. **[TBA — verdict]** *Not run. R7 (de-ranged graph) and R10 (static dump) were queued behind R6 and stopped before execution. Given that R6 shows no detection effect to decompose, their value has changed: they would now characterise the earliness effect — whether it needs the correct plan (R7) or merely a position pointer (R10) — rather than adjudicate the detection claim. Run them, or state explicitly that the earliness finding is not decomposed.*

6.3 Comparison with prior work

Under the matched protocol MistSense reaches 0.477 binary F1 against GRAPHWALK's 0.482. The paired clip-level bootstrap puts the difference at +0.005 with a 95% interval of [-0.060, +0.059]: on binary detection the two systems are statistically indistinguishable, and we claim no more than parity — and note the interval is wide enough to be consistent with the baseline being up to 0.06 F1 better. Notably, the published operating point for this baseline (0.495 F1) was obtained by sweeping the threshold on the test split; when that threshold is instead selected on validation, as ours is, the apparent margin disappears.

Two caveats bound how much this comparison carries. The baseline is our own reproduction, and we deliberately did not tune its class weighting (Section 5); its operating point is far from F1-optimal, and a better-weighted variant might exceed 0.482. And it is one recent detector, not the state of the art: we have not compared against Lee et al. (2026); Flaborea et al. (2024); Lee et al. (2024) on this benchmark.

The two systems arrive at parity by different routes. The baseline achieves its F1 through recall — it fires on 0.893 of mistake windows at 0.325 precision, and its early-detection score of 0.991 reflects that it is firing nearly always rather than firing early and correctly. GRAPHWALK is the more selective of the two (0.407 precision at 0.589 recall) and ranks windows better (AUC 0.636 vs 0.585). It also does something the baseline cannot do at all: it names a mistake type.

The cost of that selectivity should be read alongside it. Firing less often means catching fewer events: the baseline localises

Configuration	F1	P	R	EDS	MAE
Deviation flag only (no confidence gate)	0.476	0.408	0.570	0.790	61.1
Default gate (τ=0.5, AND, k=3)	0.471 [0.384, 0.548]	0.408	0.556	0.763	60.2
Validation-tuned gate (ours)	0.482 [0.392, 0.558]	0.407	0.589	0.806	60.4

Table 3: Decision-layer ablation for GRAPHWALK, all recomputed on CPU from the same saved predictions. The first row gates on the model's deviation flag alone, which is what gating on the verbalised confidence reduces to, since that field is a constant after fine-tuning.

Class	Support	Zero-shot	GRAPHWALK
`no_mistake`	16,584	0.812	0.696
`order`	4,212	0.012	0.327
`technique`	846	0.029	0.167
`preparation`	621	0.009	0.060
`measurement`	786	0.000	0.082
`timing`	556	0.000	0.150
`temperature`	89	0.000	0.000
`missing_step`	2	0.000	0.000

Table 4: Per-class F1 with window support on the test split. Support is severely imbalanced and two classes are effectively unmeasurable; `missing_step` is structurally unreachable (Section 5). These are single-run descriptive figures from one training seed (Section 7); the ordering of the low-support classes should not be read as a ranking.

61 of 61 localisable events and the zero-shot arm 61, against GRAPHWALK's 57, at comparable onset error. Which trade-off is preferable depends entirely on the application, and we do not claim ours is the better operating point — only that it is a different and more selective one, reached while also producing a typed label. We record the macro-F1 column for completeness but do not read it as a comparison — the baseline is binary by construction, so it cannot score on typing, and the gap there is definitional rather than empirical.

6.4 Where the decision signal comes from

Table 3 separates the contributions of the decision layer. Gating on the deviation flag alone — equivalently, gating on the verbalised confidence, which is constant — already recovers most of the performance, and adding the decision-token confidence with a validation-selected threshold improves binary F1 modestly. The value of the token-probability score is therefore not that it rescues the binary decision, but that it exists at all: it is the only per-window quantity in the system that is continuous, and it is what makes the threshold-free AUC of 0.636, the score smoothing, and any downstream operating-point selection possible. With the verbalised field there is nothing to rank, threshold, or smooth.

6.5 Typed prediction

Table 4 reports typed performance, and we read it conservatively, and we present typing as evidence that the *formulation* reaches something the binary literature does not, rather than as a capability claim. Macro-F1 is 0.185 (0.212 excluding `missing_step`). The system separates mistake from non-mistake well and labels ordering errors moderately, but it does not reliably name the finer execution types. Two classes cannot be measured meaningfully at all: `temperature_error` has 89 supporting windows and `missing_step` has 2. The honest summary is that GRAPHWALK is a strong binary detector with partial typing, and the typing is a direction rather than a result.

6.6 Calibration

The decision-token score ranks well above chance but is not calibrated, and we report this rather than leaving it to be inferred. Expected calibration error is 0.490 for GRAPHWALK (Brier 0.500), against 0.454 for the baseline: on this axis the baseline is the better-behaved system, and both are poor in absolute terms. The practical consequence is that the score should be used for ranking and for threshold selection — which is all our decision layer does with it — and not read as a probability of error. Calibrating it post hoc is straightforward and does not change any detection metric, since the standard mappings are monotone and our operating point is selected on validation either way; we therefore treat calibration as outside this paper's claims rather than as a contribution.

6.7 Onset localisation

GRAPHWALK detects 57 of the 61 localisable error events, with an early-detection score of 0.806 and a mean onset error of 60.4 s. The onset error is large in absolute terms; recordings in this benchmark run for several minutes, so 60.4 s is a coarse but non-trivial localisation. EDS must be read jointly with precision — a system that fires early and often scores well on it regardless of accuracy, which is exactly what the zero-shot row does.

What this operating point would mean in use. The applications motivating Section 3 — assistive guidance, training, checklists — are worth measuring the result against directly. At 0.407 precision the system raises roughly three false alarms for every two correct ones. An assistant that interrupts a cook that often is plausibly worse than one that stays silent, so we do not claim the system is deployable as an interrupting assistant. Two readings are more defensible. As a *review* tool, where a human inspects flagged windows after the fact, this precision is already useful against 0.589 recall. As an online assistant, the gap to close is precision at fixed recall, and closing it is future work rather than something the present numbers support.

7 Limitations

We state the boundaries of the claim directly.

One benchmark. All results are on CaptainCook4D, in one domain (cooking). We have not shown that graph conditioning transfers to assembly or laboratory procedures, and the

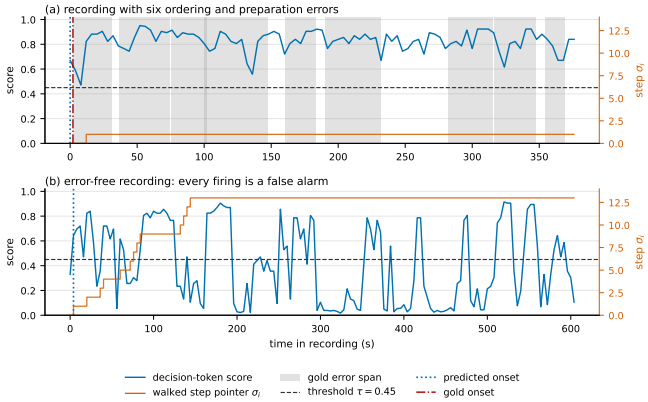


Figure 3: Two recordings judged over time, chosen to show the operating point honestly rather than favourably. **(a)** An error-dense recording: the score sits far above τ almost throughout, so the system is "correct" largely by firing continuously, and the pointer consequently stalls near the first step. **(b)** An error-free recording, where every one of the 63 firings is a false alarm, and where the pointer instead runs to the end of the graph within the first quarter of the recording and clamps — the saturation quantified above. Both panels are failure modes; with three logged recordings we cannot also show a clean success, and we prefer to show what the sample contains.

method's dependence on gold graphs means a transfer study needs a benchmark that ships them. **[[DECIDE: run a second-dataset study or scope the claim to cooking in the abstract.]]**

Graphs are given, not built. We consume gold task graphs. A deployed system would need to construct them, and graph quality would then become a confound; we have not measured the method's sensitivity to graph errors.

The conditioning is thinner than the design. As stated in Section 4, the gold graphs populate only steps and precedence edges; objects, states, and variations are empty on every window of every run.

Typing is weak. Macro-F1 0.185, with two classes unmeasurable and one structurally unreachable.

The system does not explain itself, though its interface suggests it could. The model emits a rationale field, but our supervision fills that field with a constant, and the fine-tuned model reproduces the constant. We therefore report no explanation quality and make no interpretability claim. This is a limitation of how we built the targets rather than of graph conditioning, and the benchmark's free-text error descriptions are the obvious remedy.

Scores are informative but uncalibrated. The decision-token probability ranks well above chance but its absolute values should not be read as probabilities.

Single training run. Every interval we report is an *evaluation* bootstrap: it covers which recordings were sampled, not which adapter was trained. The adapter was fine-tuned once, with a single seed, so we cannot separate the effect of the method from the variance of one training run. We judge this unlikely to overturn the headline comparison — the zero-shot to fine-tuned gap is 0.332 with a 95% interval of [0.246, 0.409], far wider than plausible seed variance at this scale — but it does bound the finer-grained claims, and the per-class numbers in particular should be read as one run rather than an expectation.

Baseline comparability. Our matched row re-tiles the baseline's per-frame probabilities onto our window grid, which is a fair alignment of outputs but not a retraining at our frame rate; and our reproduction of it carries two documented component substitutions. The baseline is also binary by construction, so the typed columns are not a comparison. Finally, parity is not superiority: on binary F1 the interval straddles zero.

8 Conclusion

Giving a vision–language model the plan — and, crucially, keeping track of where in the plan the person should be — turns a model that cannot detect procedural mistakes at all into one that detects them at 0.482 F1 on a full benchmark test split, names a mistake type, and localises its onset — reaching parity with a reproduced recent detector under a matched protocol, at a more selective operating point.

The ablation then took the paper somewhere it did not intend to go. A model trained and evaluated without any procedural graph matches this one on every detection metric, so the improvement over zero-shot belongs to fine-tuning, not to conditioning. What the graph does buy, replicating across two controls, is earliness: the same events, found sooner. And the reason the detection effect is absent is diagnosable rather than mysterious — our pointer advances once per window, saturates after the first sixth of a recording, and leaves most windows with no positional information at all.

The useful conclusion is therefore narrower and more specific than the one we set out to draw. A procedural graph, consumed this way, tells an online detector *when* to look rather than *whether* something is wrong. Whether it can do more than that is not settled here, because the mechanism we built to deliver it does not work as designed: the obvious next experiment is a pointer that advances on step completion rather than per window, evaluated against the same controls.

Ethics Statement

This work uses CaptainCook4D (Peddi et al., 2024), a publicly released benchmark of consenting adult participants recorded performing cooking tasks; we introduce no new human-subjects data and perform no re-identification. The benchmark is distributed under the Apache License 2.0, which permits the research use made of it here. Procedural mistake detection is dual-use: the same capability that supports assistive guidance and training can support workplace surveillance and productivity scoring. We note in particular that a detector with this system's precision (0.407) would produce a substantial rate of false accusations if deployed naively for individual assessment, and we regard per-worker evaluation as an inappropriate use of this work.

Use of AI Assistants

An AI coding assistant was used substantially in this work: in implementing the evaluation harness, the decision layer, the ablation tooling and the analysis and plotting scripts, and in drafting and revising this manuscript. Its use was supervised throughout, and we note two specific practices adopted because of it. First, every table and every numeric quantity appearing in the prose is generated programmatically from the saved result artefacts by a single script, so no reported number is transcribed by hand, by a human or otherwise. Second, the interpretation of the graph ablation was written and committed to version control *before* the corresponding runs completed (Section 6.2), so that the reading of the result could not be selected after seeing it — a discipline we would recommend regardless of authorship. Every citation was verified against its published record; one work asserted in our own planning notes could not be located and was removed rather than cited. The authors take full responsibility for all content, including the correctness of every claim, number and reference.

Reproducibility Statement

All reported numbers are produced by scripts in the accompanying repository from saved per-window predictions; the tables in this paper are generated programmatically from those artefacts rather than transcribed. We report the random seed, decoding configuration, complete LoRA hyperparameters (Appendix B), the exact windowing, and the validation-selected decision configuration. The prompt templates for all three conditioning variants are given verbatim in Appendix A. Code and the saved per-window predictions, from which every table in this paper is regenerated by a single script, are available at **[[ANONYMISED-CODE-URL]]**.

One reproducibility caveat we state rather than paper over: the dependency specification pins lower bounds rather than exact versions, so a fresh install resolves to whatever is current. The exact versions used for the reported runs are listed in Appendix B.

Acknowledgments

[[ACKNOWLEDGMENTS AND FUNDING]]

References

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

Alessandro Flaborea, Alessio Sampieri, Elena Cascio, Guo Yang, and Fabio Galasso. PREGO: Online mistake detection in PRocedural EGOcentric videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18483–18492, 2024.

Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Yuto Haneji, Keisuke Shirai, Taichi Nishimura, Hirotaka Kameko, Shuhei Kadowaki, and Shinsuke Mori. EgoOops: A dataset for mistake action detection from egocentric videos referring to procedural texts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2025.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Youngkyoon Jang, Brian Sullivan, Casimir Ludwig, Iain Gilchrist, Dima Damen, and Walterio Mayol-Cuevas. EPIC-Tent: An egocentric video dataset for camping tent assembly. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

Shih-Po Lee, Zijia Lu, Zekun Zhang, Minh Hoai, and Ehsan Elhamifar. Error detection in egocentric procedural task videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18655–18666, 2024.

Shih-Po Lee, Reza Ghoddoosian, Faizan Siddiqui, Enna Sachdeva, and Behzad Dariush. ESTANet: Efficient online error detection in procedural videos via prediction inconsistency. *arXiv preprint arXiv:2606.25317*, 2026.

Constantin Patsch, Yuankai Wu, Marsil Zakour, Driton Salihu, and Eckehard Steinbach. MistSense: Versatile online detection of procedural and execution mistakes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

Rohith Peddi, Shivvrat Arya, Bharath Challa, Likhitha Pallapothula, Akshay Vyas, Bhavya Gouripeddi, Jikai Wang, Qifan Zhang, Vasundhara Komaragiri, Eric Ragan, Nicholas Ruozzi, Yu Xiang, and Vibhav Gogate. CaptainCook4D: A dataset for understanding errors in procedural activities. In *Advances in Neural Information Processing Systems 37 (NeurIPS) Datasets and Benchmarks Track*, 2024.

Leonardo Plini, Alessio Sampieri, Alessandro Flaborea, and Fabio Galasso. TI-PREGO: Chain of thought and in-context learning for online mistake detection in PRocedural EGOcentric videos. *arXiv preprint arXiv:2411.02570*, 2024.

Luigi Seminara, Giovanni Maria Farinella, and Antonino Furnari. Differentiable task graph learning: Procedural activity representation and online mistake detection from egocentric videos. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. Spotlight.

Fadime Sener, Dibyadip Chatterjee, Daniel Shelepov, Kun He, Dipika Singhania, Robert Wang, and Angela Yao. Assembly101: A large-scale multi-view video dataset for understanding procedural activities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Federico Spurio, Olga Zatsarynna, Lars Doorenbos, Emad Bahrami, Gianpiero Francesca, and Juergen Gall. Post-training VLMs for video mistake detection. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2026.

Shane Storks, Itamar Bar-Yossef, Yayuan Li, Zheyuan Zhang, Jason J. Corso, and Joyce Chai. Transparent and coherent procedural mistake detection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Xin Wang, Taein Kwon, Mahdi Rad, Bowen Pan, Ishani Chakraborty, Sean Andrist, Dan Bohus, Ashley Feniello, Bugra Tekin, Felipe Vieira Frujeri, Neel Joshi, and Marc Pollefeys. HoloAssist: An egocentric human interaction dataset for interactive AI assistants in the real world. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Shengtian Yang, Yue Feng, Yingshi Liu, Jingrou Zhang, and Jie Qin. MoniTor: Exploiting large language models with instruction for online video anomaly detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A Prompt templates

All three conditioning variants share the same instruction body, the same eight-class definitions with glosses, and the same JSON output contract; they differ only in the block describing the procedure. We give that block for each, since it is the sole experimental variable in Section 6.2.

Graph-conditioned (R4, main arm).

```
PROCEDURAL GRAPH CONTEXT:
Current expected step: {current_step_id}
-- {current_step_desc}
Valid next steps: {valid_next_steps}
Required objects for current step:
{required_objects}
Expected object states: {expected_states}
Valid action variations:
{valid_variations}
```

As stated in Section 4, the last three fields are empty for every recipe in this benchmark, so the operative content is the current step, its description, and the valid next steps.

Static dump (R10).

```
RECIPE STEPS (the full procedure, in order;
the person's current position in it is NOT
given):
{all_steps}
```

Graph-free (R5, R6). The block is omitted entirely; the model receives the task name, the frames, and its own recent-judgement summary.

Common to all variants. The requested answer is a nine-field JSON object; after fine-tuning only `deviation` and `mistake_class` carry information (Section 4). The recent-history summary renders the last three windows as (observed action, deviation flag, predicted class).

B Hyperparameters and environment

Setting	Value
Base model	Qwen2.5-VL-7B-Instruct
Precision	bfloat16, no quantisation
Parameters	8.29 B (15.45 GiB of weights)
LoRA rank / scaling / dropout	32 / 64 / 0.05
LoRA targets	q,k,v,o + gate,up,down
Trainable parameters	95 M (1.13%)
Objective	completion-only LM cross-entropy
Learning rate / schedule	2×10^{-4} / cosine, 5% warmup
Batch size / grad. accumulation	1 / 8
Epochs	2
Training windows	10,000 (4951 mistake / 5049 normal)
Final training loss	≈ 0.017
Adapter size	364 MB
Window length / stride / fps	4 s / 4 s / 1
Decoding	greedy, max 512 new tokens
Global seed	42
Decision layer (val-selected)	$\tau = 0.45$, $k = 5$, $m = 1$
GPU	1× RTX PRO 6000, 96 GiB
Inference peak VRAM	17.5 GiB allocated, 18.2 GiB reserved
Inference peak host RSS	16.4 GiB
Training peak VRAM	61.7 GiB (process-level)
Training wall clock	≈3 h uncontended; ≈7 h shared
Full-split inference	7–24 h (contention-dependent)
Python	3.13.12
torch / CUDA	2.11.0+cu130 / 13.0
transformers / peft	5.8.0 / 0.19.1
accelerate	1.13.0

Table 5: Complete configuration for the reported runs. Peak-memory figures are instrumented and will be filled from the next run's log.

C Per-class support and coverage

Test-split window support by class: `no_mistake` 16,584; `order_error` 4212; `technique_error` 846; `measurement_error` 786; `preparation_error` 621; `timing_error` 556; `temperature_error` 89; `missing_step` 2. Coverage accounting for every reported run: 109 of 109 recordings evaluated, 23,696 windows, zero graphs missing, zero frames dropped, 61 localisable events, 69 non-localisable segments, and one `Other`-only step excluded from typed metrics.

D Decision-layer selection

The layer is selected by grid search on the 62-recording validation split over $\tau \in [0.30, 0.90]$ in steps of 0.025, smoothing width $k \in \{1, 3, 5\}$, gate mode $\in$ {AND, confidence-only}, chosen by validation binary F1; persistence $m \in \{1, 2, 3\}$ is then chosen by validation early-detection score at that operating point, since m cannot affect binary F1. The selected configuration is applied to the test split exactly once. Each arm receives its own selection, so no arm is advantaged by threshold placement; the graph-ablation rows deliberately reuse the main arm's frozen layer so that conditioning is the only variable.

E Reproducing the tables

Every table and every numeric quantity in the prose is generated from the saved per-window predictions:

```
python paper/gen_tables.py
cd paper && latexmk -pdf main.tex
```

The decision layer is replayable on CPU from saved predictions and reproduces a GPU run to floating-point precision, so all re-gating, bootstrap and ablation numbers can be regenerated without a GPU.